Naturaleza de los Paradigmas de Programación

Un paradigma de programación define el estilo y la estructura conceptual con la que diseñamos y escribimos software. La evolución de los paradigmas responde a la necesidad de gestionar una complejidad cada vez mayor.

Paradigma Imperativo: El foco principal del paradigma imperativo es describir cómo se debe realizar una tarea a través de una secuencia de instrucciones que cambian el estado del programa. El "estado" se refiere a toda la información almacenada en variables en un momento dado. El código es una lista explícita de comandos que le dicen a la computadora qué hacer para modificar este estado y llegar al resultado final. El control de flujo (bucles, condicionales) es un elemento central.

Paradigma Procedural: Es una evolución directa y una forma de estructurar el paradigma imperativo. En lugar de una única y larga secuencia de comandos, el código se organiza en procedimientos (también llamados funciones o subrutinas). Cada procedimiento encapsula una tarea específica. La característica fundamental es la separación entre los datos y los procedimientos que los manipulan. Generalmente, se definen estructuras de datos y luego se pasan a los procedimientos para que operen sobre ellos. Por ejemplo, tendríamos un dato factura y una función calcularTotal(factura).

Paradigma Orientado a Objetos (POO): Este paradigma representa un cambio de enfoque fundamental. En lugar de centrarse en el "cómo" (los pasos a seguir), se centra en el "quién" (las entidades responsables). La POO busca eliminar la separación entre datos y comportamiento. Lo logra mediante el encapsulamiento: agrupa los datos (atributos) y los procedimientos que los manipulan (métodos) en una sola entidad llamada objeto. El estado del programa no se manipula directamente desde cualquier parte del código; se le pide a un objeto que modifique su propio estado a través de sus métodos. Siguiendo el ejemplo anterior, no tendríamos una función externa, sino un objeto factura con un método propio: factura.calcularTotal().

Por ejemplo, pensemos en la tarea de "cambiar el aceite de un auto":

Imperativo (puro): Sería una única lista de instrucciones: "Toma la llave inglesa. Gira la tuerca del cárter hacia la izquierda. Espera a que caiga el aceite. Coloca un recipiente debajo. Retira el filtro viejo..." etc. Una larga secuencia de pasos detallados.

Procedural: Tendríamos agrupaciones (procedimientos). Uno sería "Drenar Aceite", otro "Cambiar Filtro". El mecánico seguiría usando los mismos datos (el auto, el aceite), pero la lógica estaría agrupada. Llamaríamos al procedimiento drenarAceite(auto) y luego a cambiarFiltro(auto).

Orientado a Objetos: Pensaríamos en los componentes como objetos. Tendríamos un objeto Motor. Este objeto tendría un atributo nivelDeAceite y un método cambiarAceite(). Desde el programa principal le daríamos la orden motor.cambiarAceite(). El objeto Motor es el responsable y conoce internamente los pasos necesarios para realizar esa tarea. El mecánico (el programa principal) no necesita conocer los detalles internos, solo la interfaz (cambiarAceite()).

¿Por qué la Programación Orientada a Objetos?

A medida que los programas crecen, mantener el control sobre cómo las diferentes partes del código interactúan y modifican los datos se vuelve un desafío. La POO aborda este problema al:

- **Alinear el modelo del problema con el modelo de la solución:** Nos permite modelar entidades del mundo real (un auto, una persona, una factura) como objetos en nuestro programa. Esto hace que el código sea más intuitivo y fácil de entender.

- **Fomentar la reutilización de código:** A través de conceptos como la herencia y la composición, podemos reutilizar y extender código existente de manera más efectiva, lo que mejora la productividad y la calidad del software.

- **Mejorar la calidad del software:** Al encapsular la complejidad y promover un diseño modular, la POO facilita la mantenibilidad, la escalabilidad y la fiabilidad de las aplicaciones.

Conceptos Fundamentales

Clase: Es una plantilla o un "molde" para crear objetos. Define los atributos y métodos que tendrán los objetos de ese tipo. Por ejemplo, podríamos tener una clase Auto que define que todos los autos tendrán una marca, un modelo y podrán acelerar.

Objeto: Es una instancia de una clase. Si la clase Auto es el molde, un objeto podría ser "Volkswagen Gol color rojo 2006". Cada objeto tiene su propio identificador y estado (los valores de sus atributos) pero comparte el mismo comportamiento definido en la clase.

Atributos: Son las características o datos que describen a un objeto. Para un objeto de la clase Auto, los atributos podrían ser marca, modelo y color.

Comportamientos (Métodos): Son las acciones que un objeto puede realizar. Para nuestro Auto, los métodos podrían ser acelerar(), frenar() y encender().

```
// Definimos la clase (el molde)
```

```java
public class Auto {
    // Atributos
    String marca;
    String modelo;
    int anio;

    // Comportamientos (Métodos)
    public void mostrarDetalles() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ",
Año: " + anio);
    }

    public void acelerar() {
        System.out.println("El Auto está acelerando.");
    }
}

// En otro lugar del programa, creamos y usamos objetos (instancias)
public class Concesionaria {
    public static void main(String[] args) {
        // Creamos un objeto de la clase Auto
        Auto auto1 = new Auto();
        auto1.marca = "Ford";
        auto1.modelo = "Ranger";
        auto1.anio = 2006;

        // Creamos otro objeto
        Auto auto2 = new Auto();
        auto2.marca = "Volkswagen";
        auto2.modelo = "Golf";
        auto2.anio = 2025;

        // Usamos sus métodos
        auto1.mostrarDetalles();
        auto2.mostrarDetalles();
        auto1.acelerar();
    }
}
```

nota: este código está mal intencionalmente, lo retomamos en la segunda unidad.

Lenguajes y Herramientas

La POO no se limita a un solo lenguaje. Algunos de los más importantes son Java, C++, Python, C#, y Swift. Nos centraremos en Java. Utilizaremos el Entorno de Desarrollo Integrado (IDE) VSCode, existen otros como Eclipse o IntelliJ IDEA. Estos nos proveen de herramientas para escribir, depurar y gestionar nuestro código de manera eficiente. También exploraremos el uso de sistemas de control de versiones como Git, una herramienta fundamental en el desarrollo de software profesional.

Bibliografía Sugerida para Profundizar:

- Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects.
  - Capítulo 1: Introduction to Computers, the Internet and Java.
  - Capítulo 3: Introduction to Classes, Objects, Methods and Strings.
- Barraza, O. (2006). Introducción a la programación orientada a objetos.
  - Unidad 1: Introducción a la programación.